Master's Thesis Article

Deployable Industrial Vision for Logistics

Metric Multi-Camera Warehouse Monitoring and Real-Time Conveyor Parcel Sorting on Edge Hardware

Abdullahi Adewale ATANDA[a,1] and Yang NIU[b,2]

[a] *Université Grenoble Alpes, Grenoble, France*
[b] *ISITEC International, France*

Compiled on August 27, 2026

Abstract

Industrial vision systems must produce reliable outputs that machines can use under real operating conditions. This work presents two projects built on the same detection and edge-deployment core but designed for different tasks: isiMonitor3d, a warehouse monitoring system, and IsiDetector, a conveyor sorting system.

isiMonitor3d uses one or two calibrated cameras to generate metric, identity-stable tracks and zone states for fleet control. On a dual-camera installation, it achieved 40.3 ms median and 78.1 ms 95th-percentile capture-to-publish latency, against a 200 ms target, with a 1.176 px calibration residual and 0.977 box mAP@0.5 for its best detector. A separate image-space 3×3 rack-cell detector classified storage bins as empty or filled with 0.994 box mAP@0.5. A controlled experiment showed that synthetic images cannot replace real data, although they improved recall when used as augmentation.

IsiDetector is a product deployed on a customer's sorting line. It classifies each passing parcel as carton or polybag and delivers one decision to the sorter PLC at each leading-edge crossing, timed to the sorter's control window, running on the site's CPU-only hardware. It has operated unattended through multi-hour production sessions, with the customer's receiver logging every event. Its deployed model reached 0.950 box mAP@0.5 on its validation split. However, its counting decision was not evaluated against labelled ground truth, so the results support a configuration comparison rather than a direct accuracy claim.

Together, the systems demonstrate a shared end-to-end vision core producing machine-actionable outputs for two different industrial tasks. The results show that industrial vision must be evaluated beyond detector accuracy, with latency, calibration accuracy, and operational stability determining whether outputs can be used reliably in practice.

Keywords: *industrial vision systems, multi-camera tracking, edge inference, camera calibration, synthetic data, parcel sorting, real-time event triggering, industrial deployment*

1. INTRODUCTION

1.1. Context and Problem

Warehouse automation creates a new monitoring challenge. Automated guided vehicles (AGVs) and autonomous mobile robots operate in the same spaces as people, forklifts, and moving goods, within the safety requirements defined for driverless industrial trucks in ISO 3691-4 [10]. While sensors mounted on each vehicle can monitor its immediate surroundings, they cannot answer broader site-level questions: Who is inside a danger zone? Which pallets are currently in a storage area? Has a pallet left a zone loaded or empty? Answering these questions requires infrastructure-side perception, where fixed cameras monitor the shared warehouse environment and provide information to fleet controllers and warehouse management systems. Deep-learning-based vision has increasingly been identified as an important enabler for such warehouse applications.

However, simply detecting objects in camera images is not sufficient. A fleet controller cannot directly use bounding boxes expressed in pixel coordinates. It needs metric positions in the real world, persistent object identities, and meaningful zone events such as entries, exits, and dwell times. These outputs must be generated continuously from the camera data and provided in a structured form that other systems can consume.

A similar challenge appears in automated parcel sorting. Parcels must be routed according to their type, but damaged or unreadable barcodes can prevent automatic sorting and require manual intervention. Vision can provide an alternative source of information by classifying each parcel as a rigid carton or soft polybag and sending this decision to the sorter. Here, vision is not simply a monitoring tool; it becomes part of the control loop. The classification must arrive within a specific timing window determined by the parcel's position on the conveyor, and the system must produce exactly one decision for each physical parcel, without missed or duplicated events.

Although the two applications are different, they share the same fundamental requirement: the vision system must produce reliable, machine-consumable outputs rather than images or detections intended only for human interpretation. In one case, this means positions and persistent identities; in the other, a parcel class and its crossing time. In both cases, these outputs must be continuously generated and transmitted as structured messages without human supervision.

Industrial deployment introduces additional constraints. The systems must operate in real time on the hardware available at the site, whether a single edge GPU or a CPU-only industrial computer, and often without cloud connectivity. Objects that are far from a fixed camera are particularly challenging because they occupy only a small number of pixels, making reliable detection and localization more difficult. The systems must also be practical to install and maintain by personnel who are not machine-learning specialists, tolerate failures such as a camera going offline during a shift, and adapt to site-specific layouts and object appearances for which suitable public datasets may not exist.

The objective of this work is therefore to build, deploy, and measure two complete industrial vision systems on a shared detection and edge-deployment core: a warehouse monitoring system produc-

CONFIDENTIAL: This work describes a private industrial project of ISITEC International, France.

[1]Corresponding author. Email: abdullahi.atanda@univ-grenoble-alpes.fr
[2]Industrial mentor. Email: y.niu@isitec-international.com

Master's-thesis article, Université Grenoble Alpes. Submitted to the examination committee.

ing metric tracks and zone states, and a conveyor sorting system producing per-parcel classification events. Beyond reporting their individual results, the work examines which parts of the core transfer between the two applications and which require site-specific engineering and validation.

1.2. Related Concepts and Literature

The two systems combine concepts from computer vision and industrial automation. Sections 1.2.1–1.2.5 cover the warehouse system, while 1.2.6 covers the additional concepts used by the conveyor system.

1.2.1. Camera Calibration and Projective Geometry

Camera calibration establishes the relationship between the 3D environment and its image representation. Planar-target calibration estimates camera intrinsics and lens distortion from images of a target with known geometry [1]. Fiducial markers such as AprilTag [4] and ArUco/ChArUco [5] provide reliable image-to-world correspondences and remain useful when parts of the target are occluded or blurred.

For metric localization, two main geometric approaches are commonly used. A planar homography maps points between an image and a known scene plane, making it suitable for objects that can be assumed to lie on the floor. Triangulation, in contrast, estimates full 3D position from multiple calibrated views. Homography-based methods are computationally simple but limited to a plane, whereas triangulation provides 3D information at the cost of multiple calibrated views and appropriate camera synchronization [2].

Recent industrial approaches have also explored automatic calibration without dedicated calibration targets. NVIDIA's AutoMagicCalib estimates camera intrinsic and extrinsic parameters from naturally moving objects tracked across camera views, using their trajectories and geometric relationships to derive camera projection parameters. This approach can use normal operational footage and therefore reduces the need for dedicated calibration procedures or system downtime [19].

In contrast, isiMonitor3d uses a controlled marker-based calibration procedure with ChArUco and AprilGrid targets. This provides explicit geometric correspondences and a common world reference for both single- and multi-camera localization. The resulting calibration is then shared by the system's metric localization and tracking components.

1.2.2. Multi-Camera Metric Localization and Tracking

Multi-camera tracking has been widely studied for applications such as pedestrian and vehicle monitoring. Early approaches used calibrated cameras to estimate occupancy on a common ground plane. For example, Fleuret et al. [3] proposed a probabilistic occupancy map that combines observations from multiple synchronized cameras.

Once objects have been detected, tracking-by-detection methods associate observations across successive frames to maintain object identities. Common approaches use Kalman filtering for motion prediction and Hungarian assignment for data association. ByteTrack [9] further improves association by using both high- and low-confidence detections, helping to recover objects that might otherwise be lost.

Industrial systems are beginning to adopt these ideas. NVIDIA's DeepStream multi-view 3D tracking projects per-camera detections onto the ground plane from a shared calibration, but negotiates track identities between cameras over MQTT [20]; a recent 19-camera warehouse deployment found floor-plane foot points substantially more reliable than box centers for cross-camera association [15]. Neither reports end-to-end latency. isiMonitor3d builds on the same principles while keeping one tracker in a shared metric floor coordinate system, so identities have a single owner and spatial thresholds are expressed in physical units.

1.2.3. Real-Time Object Detection Architectures

Real-time object detection increasingly focuses on balancing accuracy, inference speed, and deployment efficiency. The YOLO family established the one-stage paradigm, predicting object classes and locations in a single forward pass. Recent versions increasingly adopt end-to-end designs that reduce post-processing. YOLO26, for example, uses one-to-one prediction and removes non-maximum suppression (NMS) and Distribution Focal Loss (DFL), simplifying inference and edge deployment [17].

A parallel direction uses transformer-based detectors. DETR introduced object queries and bipartite matching to formulate detection as direct set prediction [7]. Later models such as RT-DETR/RT-DETRv2 improved convergence and efficiency, bringing transformer-based detection into the real-time range [14]. RF-DETR further explores the accuracy–latency trade-off through weight-sharing neural architecture search, making it relevant to domain-specific applications [16].

For deployment, models can be exported to ONNX and executed through hardware-specific runtimes such as ONNX Runtime and TensorRT [18]. This enables different architectures to share a common inference interface.

In isiMonitor3d, these approaches are evaluated under the same deployment conditions through a common detector interface. YOLO26 and RF-DETR variants are compared at different input resolutions, with YOLO26n-seg at 320 px selected for deployment according to the system's latency requirements.

1.2.4. Generative Synthetic Data

Synthetic data has evolved from domain randomization and rendered scenes toward photorealistic images generated by diffusion models. Latent diffusion models generate images by performing the denoising process in a compressed latent space, making high-resolution generation more practical. SDXL further improved image quality and conditioning through a larger architecture and multiple text encoders [11].

Control-based generation provides additional control over the structure of synthetic images. ControlNet allows pretrained diffusion models to follow spatial conditions such as depth, edges, or poses while preserving the model's generative capabilities [12]. Combined with segmentation, this can provide images and corresponding masks suitable for supervised training. LoRA offers an efficient alternative to full model fine-tuning by learning small, low-rank weight updates [8], making adaptation to specific object classes practical when only a limited number of real images are available. Recent promptable segmentation models such as SAM2 [13] can further automate the extraction of object masks from generated or real images.

Despite these advances, the value of combining generative models, lightweight adaptation, and automatic segmentation within a complete industrial vision pipeline remains less explored. In isiMonitor3d, the isiGen pipeline combines SDXL, ControlNet, class-specific LoRA adapters, and SAM2 to generate additional training data from a small set of real photographs. Its effectiveness is evaluated experimentally in Section 3.6.2.

1.2.5. Machine-Consumable Delivery and Edge Runtime

An industrial vision system must not only detect objects but also deliver its results reliably to the machines that use them. MQTT provides a lightweight publish/subscribe mechanism in which vision systems publish events to topics and external systems subscribe to the information they need. Its Quality of Service (QoS) levels provide different delivery guarantees for industrial communication.

Real-time video processing also requires efficient edge execution. GStreamer provides a modular framework for camera capture and

processing, while NVIDIA NVDEC can offload H.264/H.265 video decoding from the CPU. For inference, ONNX Runtime provides a hardware-independent execution layer through execution providers [18], including TensorRT for NVIDIA hardware and other backends such as OpenVINO.

In the two projects, these components form the communication and runtime layer. Camera streams are processed through GStreamer, inference runs through ONNX Runtime with TensorRT on the GPU-equipped site and OpenVINO on the CPU-only site, and results are packaged as structured JSON messages and delivered through UDP or MQTT. This allows external controllers to consume vision outputs without depending on the internal implementation.

1.2.6. *Real-Time Counting, Event Triggering, and CPU Deployment*

In the conveyor application, the required output is not an object's position but a single event for each parcel. A common approach is to detect objects in successive frames, associate them into tracks, and count a parcel when its track crosses a virtual line [6]. However, simple line-crossing methods can fail when the frame rate drops or when parcels move quickly. If a parcel moves from one side of the line to the other between two frames, the crossing may never be observed.

A more robust approach is to remember the previous position of each track. Once a parcel has been observed before the counting line and is later detected beyond it, the system triggers the event once. This reduces the risk of missed counts caused by temporary gaps between detections. The conveyor system uses this approach alongside the standard instantaneous crossing test.

For industrial sorting, the timing of the event is also important. A correct classification is only useful if it reaches the sorter within its required control window. Therefore, parameters such as the counting-line position, object reference point, and detection-to-message delay are important system-level parameters in addition to detector accuracy.

The conveyor system also targets CPU-only industrial computers, where inference efficiency is critical. Model quantization reduces computational cost by representing weights and activations with lower-precision numerical formats. The deployed model uses OpenVINO as the inference runtime and is generated from the common ONNX model used across the systems. This enables the same detection pipeline to be adapted to hardware with more limited computational resources.

2. SYSTEM A: METRIC MULTI-CAMERA WAREHOUSE MONITORING

isiMonitor3d is five stages in the order the data flows. Stages 1 (calibration) and 2 (model production) run once at commissioning and write `calibration.json` and the `.onnx` detectors; Stages 3 to 5 run continuously. Input is video from one or two fixed RGB cameras; output is metric, identity-stable object metadata, 2D floor-plane tracks (`Track2D`) continuously and 3D tracks (`Track3D`) on demand when stereo observations are available, as versioned JSON over UDP and MQTT. The five acceptance criteria of the customer specification (Section 1) are design requirements here, validated in Section 2.6.

The five modules have distinct life cycles (Fig. 1): (i) **isical**, the operator-guided calibration application that writes `calibration.json`; (ii) the offline pair **isiGen/isidet**, which generates synthetic data and trains detectors in a separate environment, exporting only ONNX; (iii) **isistream**, the perception producer (acquisition, decoding, object detection, pose); (iv) the **backbone metric engine**, owning all geometric reasoning and identity; and (v) **isicomms**, providing MQTT publication and a REST gateway for polling clients such as AGVs. Because the life cycles differ (once per deployment, offline on retraining, continuous), each module deploys, updates and restarts on its own. Table 6 summarizes the five stages.

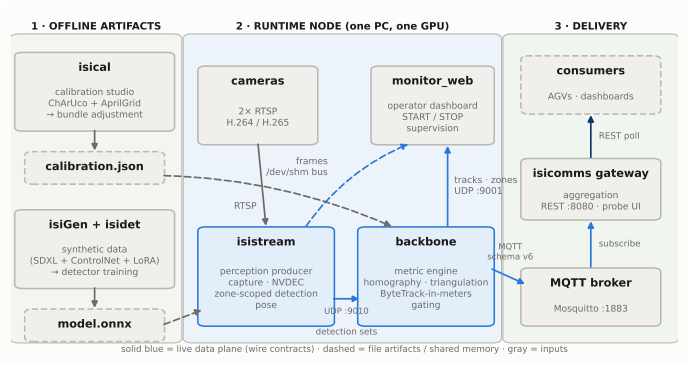

Figure 1. System A architecture in three parts: offline artifact production, the runtime node, and delivery to consumers, connected by file artifacts and two frozen wire contracts.

At runtime, Stages 3 and 4 are two processes connected through a loopback interface ("Direction 1"): perception is GPU-intensive and throughput-oriented, metric reasoning CPU-light and latency-sensitive, and running both in one Python process caused contention between the interpreter lock and the inference thread pools, quantified in Section 4.1. isistream sends one `DetectionSetMessage` per camera to the backbone metric engine over a dedicated UDP loopback port (default 9010), a configuration called *points mode*; the engine works on detection metadata alone and imports neither CUDA runtimes nor inference libraries.

Each message carries four pieces of runtime information: the frame `capture_ts`, the single latency reference propagated downstream (cached detections are re-emitted with the current timestamp when the motion gate of Section 2.3 suppresses inference); an explicit-empty heartbeat, so silence means a camera fault rather than an empty scene; a per-camera monotonic sequence counter, so UDP loss shows up as measurable gaps; and a configuration fingerprint exposing calibration, model or zone mismatch between producer and engine.

Decoded frames are also distributed through a decode-once shared-memory frame bus, one double-buffered seqlock segment per camera (`/dev/shm/isi3d_frame_<cam>`) with lock-free readers: one RTSP session and one decode per camera, and every consumer sees the exact pixels the perception pipeline processed.

Modules communicate only through two stable interfaces, the backbone UDP/JSON schema (Section 2.5) and the isicomms MQTT-in/REST-out interface: they share messages, not code, and evolve by extending the schema. Inside the backbone, extensibility is limited to five abstract plugin interfaces (`FrameSource`, `Detector`, `Tracker`, `Triangulator`, `MetadataSink`) that self-register through a registry, a fixed set pinned by a unit test; everything else, projection, fusion, gating, stabilization, is concrete, so substitution exists only where algorithmic or hardware diversity is real.

2.1. Stage 1: Site Geometry and Operator-Guided Calibration

Stage 1 is performed once when a new camera rig is installed. Its purpose is to determine the geometry of each camera and save it in a single `calibration.json` file. All later stages use this same file, ensuring that the system has one consistent geometric reference.

Calibration is designed for site operators and requires only printed planar targets. No surveying equipment or computer-vision expertise is required. The process has three main steps: intrinsic calibration, relative pose estimation, and floor-plane anchoring.

First, each camera is calibrated independently using handheld images of a ChArUco board. In the deployed system, 25 valid images were captured per camera. This step estimates the camera intrinsic matrix (**K**) and distortion coefficients (**D**). Capture quality is automatically checked using the number of detected corners, image sharp-

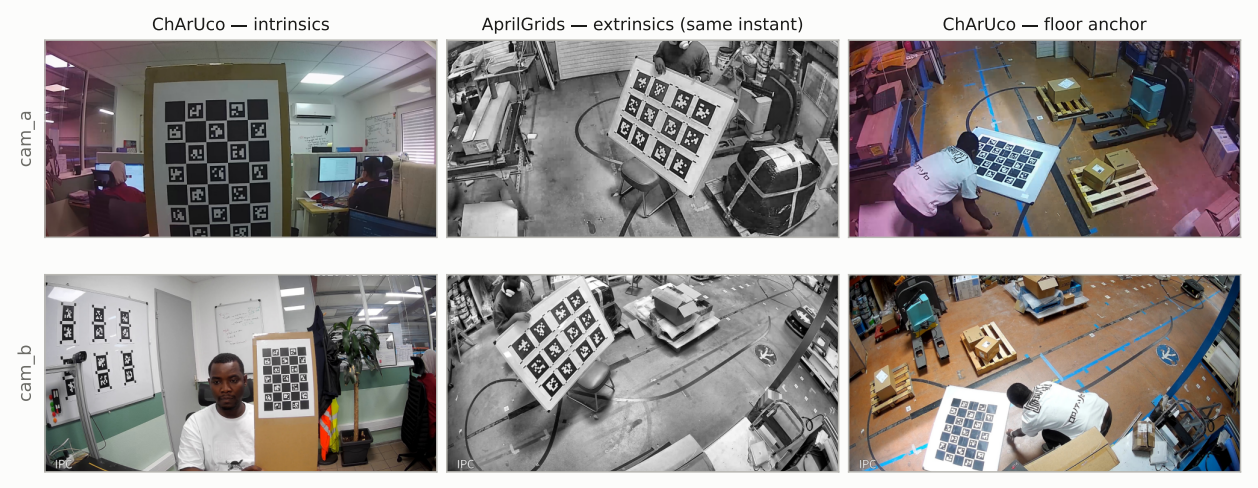

Figure 2. Calibration materials and verification: ChArUco and AprilGrid boards as deployed, and the bundle-adjustment viewer with reprojection errors.

ness, camera motion, and the difference between successive board positions.

Second, the relative camera poses are estimated from synchronized image pairs of an AprilGrid target placed across the shared field of view. The intrinsic parameters are kept fixed during this optimization, preventing errors in the intrinsic calibration from being absorbed into the camera poses. Eight synchronized image pairs were sufficient for the deployed installation. The resulting poses describe the cameras in a common coordinate system but do not yet define their position relative to the warehouse floor.

Finally, the coordinate system is anchored to the floor. The ChArUco board is placed at several positions on the floor, with eight placements used per camera in the deployed system. Each placement provides a 3D pose using PnP, and the resulting planes are combined using RANSAC to estimate the warehouse floor as $Z = 0$. The first placement defines the origin and axes of this world frame. Multiple placements make the estimate more robust to an individual poorly positioned target.

For single-camera installations, the floor homography can instead be obtained directly from operator-measured pixel-to-floor correspondences. At least four non-collinear points are required, while five or more are recommended to provide an overdetermined estimate. The calibration is rejected if the maximum floor-position error exceeds 0.10 m.

The final calibration file stores, for each camera, the intrinsic parameters ($\mathbf{K}$, $\mathbf{D}$), the pose ($\mathbf{R}$, $\mathbf{t}$), the floor homography ($\mathbf{H}$), and the projection matrix ($\mathbf{P}$). During runtime, image points are first undistorted using $\mathbf{K}$ and $\mathbf{D}$, then mapped to metric floor coordinates using $\mathbf{H}$. The projection matrix $\mathbf{P}$ is used when a 3D world point must be projected into the image.

Before calibration is accepted, the system checks the reprojection error. The deployed thresholds are 0.5 pixels for single-stage calibration and 2.0 pixels for the joint extrinsic optimization. A 3D visualization of the cameras, calibration targets, and reprojection errors provides the final operator check (Fig. 2).

The resulting `calibration.json` is then used unchanged by Stage 3 to project monitoring zones into the camera views and by Stage 4 for all metric geometric measurements. This establishes the "one calibration, two queries" principle used throughout the system.

2.2. Stage 2: Data Preparation and Detector Models

Stage 2 converts annotated site images into the detector models used by Stage 3. It produces ONNX models that are loaded directly by the deployed perception pipeline. Both the warehouse and rack datasets are prepared through isiGen, the shared dataset pipeline, ensuring a common process for annotation, splitting, augmentation and dataset export.

isiGen: shared dataset pipeline. isiGen provides the common dataset-production workflow used by both systems. It consists of ten plugin steps covering data curation, automatic object detection, SAM2 mask generation, control-map generation, captioning, LoRA training, image generation, quality filtering, and dataset export. The complete pipeline therefore supports both conventional real-image dataset preparation and synthetic-data generation.

The generative path is specific to System B and was used to produce synthetic parcel images. It is described separately in Section 3.2, where its contribution is evaluated through the synthetic-data ablation. The System A detector results reported in this section were trained exclusively on real images; synthetic images were kept separate from the main training corpus.

Warehouse-object detector. The first detector identifies three warehouse object classes: `palette` (pallet), `carton`, and `polybag`. The dataset contains 5,540 training images and 1,049 validation images, all captured from real sites. The models used are YOLO26-seg and RF-DETR-seg. They are fine-tuned using isidet in a separate Conda environment and exported to ONNX with opset 17, preserving the raw prediction head. This isolates training dependencies from the deployed perception stack and allows the same model artifact to be used by Stage 3. Detector performance is reported in Table 1.

Rack-cell detector. The second detector addresses a different task: determining whether each rack cell is empty or filled. Each shelf cell is treated as an individual image and labelled as `empty_box` or `filled_box`. Images are extracted from recorded rack sequences using the same framing as inference: 320 px letterboxing with an 8 % margin. The dataset contains 3,320 training images, including 284 label-free empty-shelf backgrounds, and 155 validation images.

Offline augmentation includes horizontal flipping, brightness and contrast changes, and Gaussian blur, while the original images are retained. The dataset is split by source frame, ensuring that augmented versions of the same frame cannot appear in both training and validation sets.

A YOLO26n detector with 2.4 M parameters and 5.2 GFLOPs (fused) is fine-tuned at 320 px for 100 epochs using AdamW, batch size 32, and seed 0, then exported to ONNX.

During export, a specific decoding ambiguity was identified for the two-class rack model. Its NMS-free raw output uses rows of the form $[x_1, y_1, x_2, y_2, score, class]$. Because two classes produce six values per row, this can be confused with an alternative anchor-based layout that also has six values. The decoder therefore identifies the format from the structure of the output rather than its width alone: end-to-end outputs contain a few hundred rows with an integer class column,

Table 1. Detectors trained in one trainer and exported through one path, across both systems. Rows are trained and evaluated on different corpora, tasks, and validation splits, so they are not comparable to each other as accuracies.

Sys.	Model	Task	In	Checkpoint	Validation set	Box mAP@0.5
A	YOLO26l-seg	seg	640	epoch 154 of 172	pallet3, 1,049 img (1,436 inst.)	**0.977**
A	RF-DETR medium-seg	seg	432	best EMA, epoch 23 of 41	pallet3, 1,049 img (1,436 inst.)	0.973
A	YOLO26n-seg (deployed, zone crops)	seg	320	epoch 198 of 200	crop384, 1,033 crops	0.974
A	YOLO26n (rack cells)	det	320	epoch 75 of 100	rack cells, 155 img (137 inst.)	0.994
B	YOLO26m-seg	seg	416	epoch 193 of 200	parcels, 242 img (389 inst.)	**0.970**
B	RF-DETR medium (best of three runs)	seg	unrec.	best EMA, epoch 10	parcels, 242 img (389 inst.)	0.967
B	YOLO26n-seg	seg	416	epoch 134 of 184	parcels, 242 img (389 inst.)	0.955
B	YOLO26n-seg (deployed)	seg	320	epoch 166 of 196	parcels, 242 img (389 inst.)	0.950

whereas anchor-based outputs contain thousands of rows with fractional confidence scores. The three-class warehouse detectors do not have this ambiguity.

The final ONNX models produced by Stage 2 are passed unchanged to Stage 3, where they are executed using ONNX Runtime.

2.3. Stage 3: Perception

Stage 3 is the online perception stage. It takes RTSP video from one or two fixed cameras, together with the calibration from Stage 1 and the detector models from Stage 2, and produces one `DetectionSetMessage` per camera over the UDP loopback interface. Its latency and throughput are evaluated in Section 2.6.4.

2.3.1. Video Capture, Runtime Loop, and Inference

isistream owns the video path from camera acquisition to the detection messages the rest of the system consumes. Each camera is read over RTSP (TCP) through a GStreamer pipeline; the codec is detected at startup and the matching H.264 or H.265 depayloader selected, so cameras with different codecs operate together. Decoding uses NVIDIA NVDEC with GPU color conversion and resizing where available, with a software fallback. The pipeline keeps only the newest decoded frame, so stale frames are dropped rather than queued, and each frame is stamped with `capture_ts` at the appsink, the temporal reference for the whole pipeline and all latency measurements; relative to the physical scene it lags by the RTSP jitter buffer (about 100 ms) plus decode time.

A paced loop processes the newest frame from each camera: zone crops from all cameras go through one batched object-detection inference, and person-pose estimation runs on the full image every n-th iteration (n configurable). A `DetectionSetMessage` is published for every camera even when nothing is detected; a frame that goes stale before processing is discarded and its camera treated as temporarily unavailable.

Motion gate. Warehouse scenes are often static, so inference is skipped when nothing moves: a 32×32 grayscale comparison marks a frame as changed when more than 2 % of pixels differ by at least 15 levels. The test runs per zone for object detection and on the full frame for pose, inference is forced at least every 2 s, and skipped ticks re-emit the previous detections with the new `capture_ts`, keeping the output stream continuous.

Detection models. Object detection uses YOLO26-seg and RF-DETR-seg models exported to ONNX and run through ONNX Runtime, so the same file serves the development machine and Jetson production hardware through interchangeable execution providers: TensorRT in the measured configuration (engines cached on disk, batch sizes bucketed at 1–32 to bound engine builds; an optional SAHI mode tiles unusually large zones), CUDA as a fallback, and an OpenVINO plugin for Intel CPUs. Person pose uses a YOLO11-pose model; the ankle midpoint of each person's 2D keypoints becomes the foot point Stage 4 consumes. Masks are polygonized and packed with boxes, foot points, and keypoints into the per-camera `DetectionSetMessage`, rack states travel separately, every message carries its `capture_ts`, and no pixels leave Stage 3.

2.3.2. Zone-Scoped Detection

Object detection is restricted to operator-defined floor regions, while pose estimation keeps the full image. Zones are polygons in metric floor coordinates, drawn on the floor map or by clicking points in a camera view (converted through the Stage 1 calibration), each with a base height z_{base}: 0 m for floor zones, higher for raised surfaces. At runtime each polygon is projected into the camera image and extended vertically from z_{base} to $z_{\text{base}} + 2$ m so loaded pallets stay inside the region; zones outside a camera's field of view are ignored.

All plane geometry runs through one primitive: the undistorted pixel ray intersected with the plane $Z = z_{\text{base}}$, which for $z_{\text{base}} = 0$ matches the homography exactly. It drives zone projection, in-zone membership, and the zone decisions of Stage 4, while published track positions stay raw rather than snapped to the plane. Visible crops from all cameras are letterboxed to the detector input size (384 px by default; 320 px in the deployed system and all experiments of Section 2.6) and run in one batched inference, with detections mapped back to full-image coordinates. Cropping magnifies distant zones, so small objects get better effective resolution than in full-frame detection.

Zone-crop training data. Because the detector sees letterboxed crops, the deployed YOLO26n-seg model was trained in the same format: 5,731 training and 1,033 validation crops (384 px object-centred windows with the inference-time gray fill and enhancement), plus 89 empty-support crops and 356 photometric variants added after an early deployment detected the empty pallet support as a pallet (performance in Table 1). Overlapping zone crops can detect the same object twice, so a per-camera, per-class deduplication keeps only the highest-confidence detection; with no zones configured, the system runs pose-only.

Cross-camera zone twins. A zone defined once in metric coordinates projects independently into each camera, giving two views of the same physical region. When a forklift or load occludes one view, the twin keeps producing detections, and Stage 4 merges the two streams or falls back to the survivor. Twins are regenerated whenever the calibration changes; the cameras share only the zone geometry, never detections or masks.

2.3.3. Rack Cells

Rack storage requires a separate approach because bins are located at different heights and do not lie on the warehouse floor. Rack processing therefore remains entirely in image space and does not use the floor geometry or homographies (Fig. 3).

An operator defines a rack by selecting a camera and clicking its four outer corners. The system then subdivides the quadrilateral into a configurable grid, 3×3 by default. Individual cells can be moved, resized, and rotated to account for perspective distortion.

Rack definitions are stored separately in `config/etagere.yaml`. This keeps rack processing independent from floor zones and means that racks can be configured even before camera calibration is available.

At runtime, each cell is transformed to an upright crop with an 8 % margin and letterboxed to 320 px, matching the training data. All

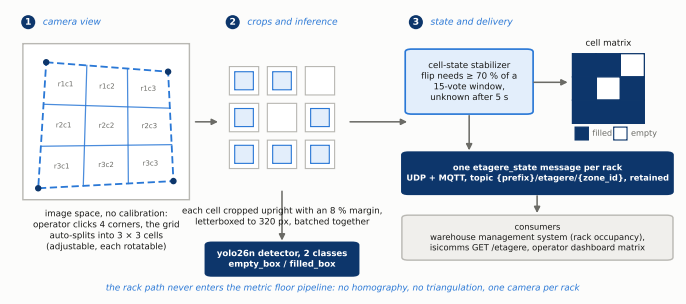

Figure 3. The rack-cell path: a 3 × 3 operator-drawn grid, batched per-cell classification, and one retained state message per rack.

cells are processed together in one batched inference. A cell is classified as `filled_box` or `empty_box` when the highest-confidence detection exceeds 0.3; otherwise its state is unknown. Rack inference is limited to 2 fps by default, since shelf contents normally change much more slowly than moving objects.

Each rack produces an `etagere_state` message on the same loopback interface as the object detections. Rack processing is independent of floor zones and can continue even when no floor zones are configured.

The metric engine stabilizes these raw verdicts cell by cell: a held state flips only when a challenger wins at least 70 % of a 15-vote window (11 of 15), an unknown verdict never votes (so a hand passing over a bin cannot flip a cell), and a cell silent for 5 s decays to unknown. A rack is republished whenever any cell changes and every 5 s as a heartbeat. Rack cells never enter the homography or triangulation paths and carry no `track_id`: they are state to be debounced and forwarded, not geometry.

2.4. Stage 4: Decision Layer, Metric Geometry and Identity

Stage 4 is the metric engine: it turns the per-camera detection messages of Stage 3 into tracks and operational states, the continuous `Track2D` stream, the on-demand `Track3D` stream, and the retained zone states. Two invariants organize it (Fig. 4): both geometric queries read the same `calibration.json`, and identity is created exactly once, by the 2D tracker; the 3D pipeline only adds a third coordinate to existing tracks. Runtime behaviour and geometric verification bounds are evaluated in Section 2.6.4.

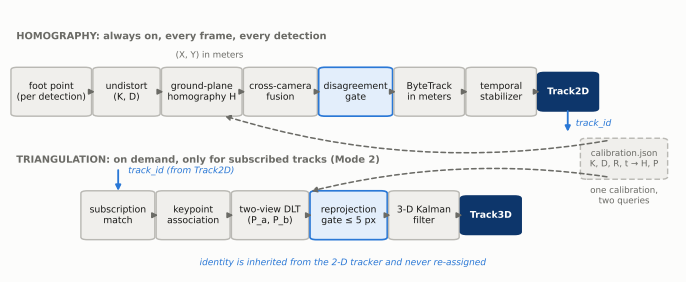

Figure 4. The geometric pipeline: always-on homography to `Track2D`, on-demand triangulation to `Track3D`, one shared calibration and identity space.

2.4.1. Frame Synchronization and Operating Modes

Detection messages are synchronized on their `capture_ts`: when the oldest buffered frame from every camera falls within a 33 ms tolerance (one frame at 30 fps), they form one multi-camera observation. The camera count selects the operating mode: with one camera only the homography pipeline runs; with two, homography runs continuously and triangulation on request. If a camera fails in stereo mode, the engine waits 100 ms, drops stale frames, and continues from the survivor: `Track2D` never stops, `Track3D` suspends, and identities survive the outage and the recovery. If every camera degrades at once, as in a start-up stall, one camera is re-admitted to aligned pairing every 2 s so synchronization can re-form.

2.4.2. Metric Localization: Homography and Triangulation

Localization always starts on the floor. For each detection only the floor-contact point is projected, the bottom-centre of the bounding box for objects and the ankle midpoint for persons. The point is undistorted with $(\mathbf{K}, \mathbf{D})$ and mapped through the calibrated homography,

$$\begin{pmatrix} X \\ Y \\ 1 \end{pmatrix} \propto \mathbf{H}\,\tilde{\mathbf{p}}, \qquad \tilde{\mathbf{p}} = \Pi^{-1}(\mathbf{u};\, \mathbf{K}, \mathbf{D}). \tag{1}$$

When several cameras see the same object, including the two views of a twinned zone, floor positions are matched by Euclidean distance with Hungarian assignment (0.8 m for persons and pallets, 1.6 m for forklifts) and merged by averaging. A tighter per-class agreement gate (0.4 m / 0.8 m) rejects fusions whose views disagree and keeps the higher-confidence observation, the "fail honestly" principle applied inside the 2D path. On the deployed rig the pallet thresholds are widened to 1.0 m and 0.8 m to absorb the loading-platform parallax described in Section 2.3.2.

Triangulation adds height on demand: it runs only for tracks matching declarative subscription rules (object class, minimum observing cameras, zone membership, rate limits), so its cost scales with consumer demand rather than scene complexity. For a subscribed track, the per-camera observations of the same synchronized frame are recovered and triangulated with the standard Direct Linear Transform,

$$[\mathbf{p}_i]_\times \mathbf{P}_i \mathbf{X} = \mathbf{0}, \tag{2}$$

solved jointly by singular value decomposition. A point is accepted only if its maximum reprojection error across the contributing views stays within a gate: 5 px by default (the specification permits 5–8 px), opened to 60 px on the deployed rig, where the two views' box-derived foot points of large objects are not the same physical point and correspondence, not calibration, dominates the residual. Accepted points feed a constant-velocity 3D Kalman filter indexed by the `Track2D` identifier, so `Track3D` carries exactly the 2D identities plus metric height. With exactly two cameras the DLT is exactly determined, which limits what reprojection error alone can detect; the upstream agreement gate mitigates this.

2.4.3. Metric-Space Tracking

Fused observations feed a modified ByteTrack running directly in metric coordinates: all cameras share one global tracker and no cross-camera identity reconciliation exists. Each track holds a constant-velocity Kalman filter with state $[X, Y, v_X, v_Y]$. Observations are split into high- and low-confidence sets (thresholds 0.5 and 0.1), and association runs in two Hungarian stages on the Mahalanobis distance: first all active tracks against high-confidence detections, then the remainder against low-confidence ones. Unmatched high-confidence observations start new tracks, and identifiers are never reused, so identity continuity survives temporary single-camera operation. Class labels are stabilized by majority vote over a sliding window, and tracks can be held back until a confirmation period passes; the result is the continuously published `Track2D` stream.

2.4.4. Zone and Pallet State

Pallet occupancy is estimated by a rule-based module from two cues with different failure modes, image overlap and metric floor margin, and stabilized by the same majority vote. The zone-level verdict consumed by the WMS comes from a dedicated pallet-state manager fed by detection evidence rather than the track list: a detection

from any camera whose foot projection lies inside the zone polygon (±0.15 m) proves presence, occupancy is resolved across cameras by "loaded wins" and per-class counts by maximum, and the resulting state (`no_data`, `no_palette`, `palette_empty`, `palette_loaded`) enters after 2 evidence frames, leaves after 15 absent frames, and is published as a `decision` field on the retained zone state (Section 2.5). Track-level zone membership keeps its own entry/exit hysteresis (3 frames in, 15 out).

`Track2D`, `Track3D`, and the retained zone state leave this stage as schema-versioned JSON envelopes, which Stage 5 carries to consumers unchanged.

2.5. Stage 5: Delivery and Deployment

Stage 5 is the communication layer of the system. It takes the JSON messages produced by Stage 4 and delivers them to local modules, remote subscribers, and external systems such as AGV controllers. Its runtime behaviour is evaluated in Section 2.6.4.

2.5.1. Communication Interfaces

The isicomms service provides the external communication layer through MQTT and REST. Local components use UDP/JSON for low-latency communication, while MQTT provides broker-based distribution to remote and multi-node consumers.

MQTT topics follow `isiMonitor3D/v1/<node>/<message-family>`, with separate topic families for tracks, zones, racks, proximity events, diagnostics, and configuration. State-bearing topics use QoS 1 with retained messages. The MQTT topic version is independent of the payload schema version, allowing message routing and payload formats to evolve separately. The MQTT sink reconnects automatically using a 1–30 s exponential backoff. If the broker becomes unavailable, delivery is degraded but perception and geometric processing continue unaffected.

For systems that cannot use MQTT, such as some AGV fleet controllers, the same service provides a REST API on port 8080, with optional token authentication. The API exposes nodes, zones, tracks, passing events, and rack occupancy, with filtering by node, object class, zone, and 2D/3D track type. A commissioning probe also exposes the active MQTT topic hierarchy.

These interfaces isolate external consumers from the internal perception and geometry pipelines, allowing industrial systems to integrate with the monitoring results without modifying the core processing stages.

2.5.2. Runtime and Deployment

Development and evaluation are performed on a Linux/WSL2 workstation with an NVIDIA RTX 5070 (12 GB) using Python 3.10, OpenCV 4.13, GStreamer 1.28, and ONNX Runtime 1.23.2 with TensorRT 10.16 and CUDA 12.9.

The production target is an NVIDIA Jetson Orin NX 16 GB (Seeed reComputer J4012). Because the perception system uses framework-independent ONNX models through ONNX Runtime, the same models and application code can be transferred from the workstation to the Jetson with only the runtime environment changing.

In deployment, the perception producer and metric engine run as two independent systemd services. They share the same configuration, start automatically, recover after failures, and operate entirely locally without requiring a cloud connection.

Stage 5 therefore completes the pipeline by converting the internal perception and tracking results into stable interfaces that can be consumed by monitoring software, WMS/FMS systems, AGV controllers, and other external applications.

2.6. Results for System A

This section reports the main results for System A in pipeline order: detector accuracy, rack-cell performance, calibration accuracy, and live runtime.

2.6.1. System A: Pallet Corpus

The `pallet3` corpus contains 5,540 training and 1,049 validation images for three classes: `palette` (pallet), `carton`, and `polybag`. A provenance audit confirmed that all images are real photographs. No independent test set was available; therefore, all reported accuracy is based on the validation split.

The detector results are summarized in Table 1. The independently evaluated YOLO26l-seg model achieved 0.977 box mAP@0.5, 0.948 box mAP@0.5:0.95, 0.972 mask mAP@0.5, and 0.921 mask mAP@0.5:0.95, with 10.3 ms/image inference at batch size 8; its per-class breakdown is given in Table 2. RF-DETR medium-seg achieved per-class box AP values of 0.916 (`palette`), 0.905 (`carton`), and 0.971 (`polybag`).

All configurations exceeded the acceptance target of mAP@0.5 $\geq$ 0.90, with overall values of 0.973–0.977. The deployed 320 px model achieved 0.974.

The pallet empty/full occupancy KPI was not evaluated because no labelled dataset was available for this task.

2.6.2. System A: Rack-Cell Corpus

The rack detector classifies individual shelf cells as `empty_box` or `filled_box`. Its validation set contains 155 original images from the recorded rack sessions. Training required 0.939 h, while inference took 0.5 ms per cell when batched and 4.2 ms individually on the RTX 5070.

These results characterize the evaluated rack only. Since training and validation originate from the same recording sessions, they do not demonstrate generalization to other racks, cameras, or sites.

The detector was not enabled in the live production configuration because on-rig verification was still pending. Consequently, no live rack accuracy, throughput, or end-to-end latency is reported.

2.6.3. Stage 1: Calibration Accuracy

The deployed `c1` rig was recalibrated on 6 August 2026 after camera movement degraded the previous July calibration. The updated solve used 25 ChArUco images per camera, 8 synchronized AprilGrid pairs, and 12 floor placements across the two cameras.

The calibration passed the acceptance gate, retaining 768/768 board points with a joint extrinsic consensus residual of 1.176 px per camera (Table 3). Reprojection-error quantiles were 0.09 / 0.69 / 0.97 / 1.32 / 3.80 px (min / Q1 / median / Q3 / max). The previous July calibration had a residual of 1.621 px.

The updated calibration therefore satisfied the $\leq$ 2 px acceptance criterion. Because the new solve defined a new world origin, the existing zone polygons had to be redrawn.

2.6.4. Stages 3–5: End-to-End Runtime

Live performance was measured for 310 s, using 61 diagnostic heartbeats covering rolling windows of 2,048 published messages. The production configuration used motion gating, pose stride 1, TensorRT, zone-scoped detection, and two active cameras, with the dashboard, MQTT broker, and gateway all running (Fig. 6; the rack-cell view in Fig. 7).

The system processed approximately 25.8 fps aggregate (>18,000 frames), with approximately 13.4–13.8 fps per camera. Capture-to-publish latency measured 40.3 ms at p50, 78.1 ms at p95, and 94.0 ms at p99 (Fig. 5), meeting the 200 ms acceptance limit with a 2.6× margin. Both cameras remained available throughout the measurement.

Table 2. YOLO26l-seg per-class validation accuracy (independent re-evaluation).

Class	Images	Instances	Box P	Box R	Box mAP@0.5	Box mAP@0.5:0.95	Mask mAP@0.5	Mask mAP@0.5:0.95
palette	808	1,047	0.973	0.945	0.989	0.933	0.974	0.895
carton	74	174	0.909	0.948	0.960	0.943	0.960	0.900
polybag	186	215	0.967	0.958	0.983	0.969	0.980	0.967

Table 3. Deployed-rig calibration reprojection RMS.

Quantity	cam_a	cam_b	Applicable gate
Intrinsic-stage reprojection RMS (25 shots/cam)	0.603 px	0.451 px	none (feeds the fixed-K extrinsic solve)
Joint extrinsic consensus reprojection RMS	1.176 px	1.176 px	$\leq$ 2.0 px (assembly gate)

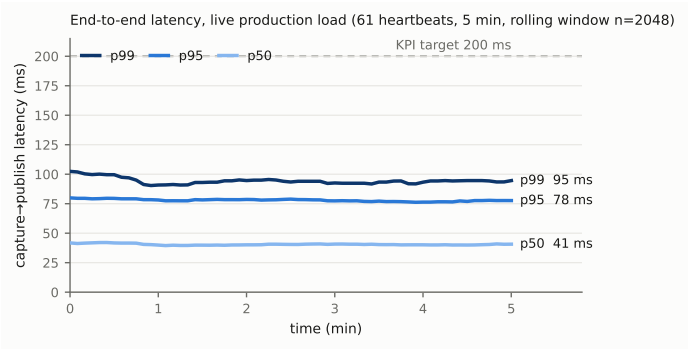

Figure 5. Live capture→publish latency: per-heartbeat p50/p95/p99 over 310 s, against the 200 ms KPI.

An earlier configuration, before ingest downscaling, pose-input reduction, and motion gating, measured 77 ms p50 / 126 ms p95. These values are provided only for comparison; Fig. 5 reports the final configuration.

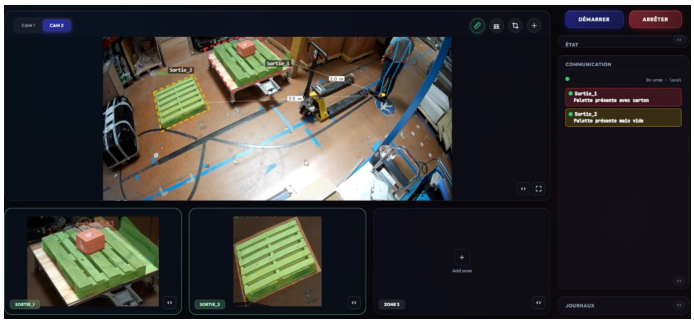

Figure 6. The operator dashboard in live operation: camera view with zone polygons, a tracked person with metric proximity distances, and per-zone occupancy cards fed by the MQTT zone state.

3. SYSTEM B: CONVEYOR PARCEL CLASSIFICATION AND SORTER TRIGGERING

System B monitors a single conveyor using a fixed overhead camera (Fig. 9). It classifies each parcel as carton or polybag and sends exactly one datagram when the parcel's leading edge crosses a predefined trigger line (Fig. 10). The sorter PLC uses this event to route the parcel without polling or human intervention.

The system was developed for Celio, a French clothing retailer. Its sorter normally relies on barcode identification; System B provides a backup classification when barcode reading fails. Polybag detection is the main challenge because the material is deformable, glossy, and sensitive to specular reflections from the overhead lighting.

Figure 7. Rack-cell detection on the dashboard: grid overlay on the camera view, per-cell detections, and the published cell matrix.

Three constraints guided the design. First, the system must run on the site's available computer, with or without a GPU, so the software cannot depend on CUDA. Second, the detector must remain reliable under polybag glare. Third, deployment and operation must be accessible to non-ML engineers, with configuration exposed through a web interface and deployment automated by scripts.

The PLC interface is defined by a timing window rather than a strict vision latency target: it accepts a camera event between 600 and 1,100 ms after its reference instant, providing approximately 500 ms of tolerance. The vision system therefore focuses on generating a repeatable trigger at the correct point in the parcel's movement, while the PLC absorbs the remaining timing offset.

System B reuses most of the validated components from System A, including the training pipeline, YOLO26-seg and RF-DETR-seg, raw-head ONNX export, interchangeable inference backends, ByteTrack, and Docker-based GPU/CPU detection. The following sections focus only on the components that differ for the conveyor-triggering task (Fig. 8).

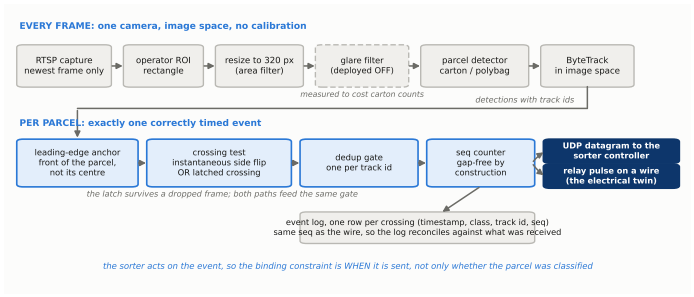

Figure 8. System B, from a conveyor frame to one correctly timed sorter event: the per-frame image path above, the per-parcel decision layer below.

3.1. Stage 1: Site Geometry, Region of Interest and Trigger Line

System B uses a simpler spatial configuration than System A because its task does not require metric localization. The camera is fixed above the conveyor, and no camera calibration is required.

During commissioning, the operator defines a single region of interest (ROI) by drawing a rectangle around the conveyor on the live camera view. The ROI removes static areas outside the belt and reduces the amount of image processed by the detector. It is resized to the detector input using area interpolation, with the deployed detector operating at 320 px.

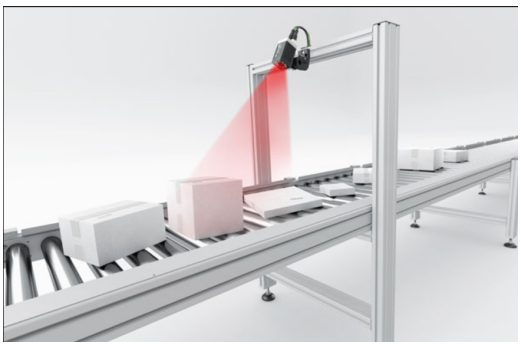

Figure 9. System B acquisition geometry: one fixed overhead camera above the conveyor (illustration).

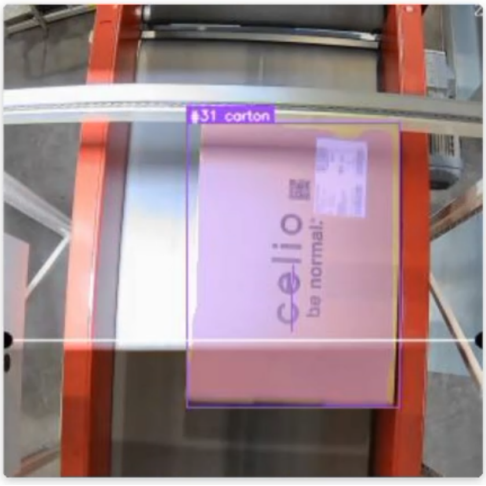

Figure 10. Live System B detection: a tracked carton with instance mask, class, and track identifier at the counting line.

A single horizontal trigger line defines the point at which a parcel must generate an event. In the deployed configuration, the belt moves from top to bottom and the line is positioned at 71 % of the ROI height. This position was selected from the line-placement experiment reported in Section 3.6.3.

The system does not use the parcel centre to determine the crossing. Instead, it tracks the leading edge of the parcel. Consequently, the position of the trigger line and the definition of the leading-edge anchor must be commissioned together: moving the line changes the point at which the parcel is considered to have crossed.

Thus, the complete spatial configuration consists of only one ROI, one trigger line, and the belt direction. This avoids the calibration and metric-coordinate requirements of System A and makes commissioning suitable for non-ML site operators.

3.2. Stage 2: Data Preparation and Detector Models

System B was trained using parcel images from the customer's conveyor, complemented by synthetic data generated with the shared isiGen pipeline.

Synthetic data. The synthetic-data workflow starts from curated real parcel photographs. Object masks are generated using SAM2, while monocular depth estimation provides depth control maps. A class-specific LoRA adapter with rank 16, trained from 53 real photographs, is combined with a frozen Stable Diffusion XL model and a depth ControlNet.

The generation process uses procedurally created scaffolds containing the control map and corresponding object mask. Because the mask is known during generation, synthetic images automatically have segmentation annotations (Fig. 11). For the black-polybag class, 500 scaffolds produced 500 synthetic images, which were subsequently CLIP-filtered and exported in YOLO-seg format. Synthetic data was particularly useful for the difficult polybag class, whose glossy and deformable surface makes real-image coverage more difficult to obtain.

Real parcel corpus. The real corpus was collected from the customer's conveyor and sampled at 2 fps from recorded sequences. Polygon annotations produced a master dataset of 1,208 images containing 1,788 instances: 741 cartons and 1,047 polybags. A seeded 80/20 split was used. Two augmented versions of each training image were then generated, giving 2,898 training images with 4,197 instances and a validation set of 242 original images with 389 instances. The augmentations reproduce common conveyor failure conditions: brightness and contrast changes, motion blur, sensor noise, JPEG compression artefacts, and hue shifts. The validation images were not augmented.

Model training. All evaluated models perform instance segmentation. YOLO26-seg experiments used AdamW, batch size 16, cosine learning-rate scheduling, seed 0, and up to 200 epochs. Nano models were evaluated at 320 and 416 px, while medium models were evaluated at 416 and 512 px.

RF-DETR-seg was also evaluated at 416 px with batch size 2 and EMA enabled. However, the corresponding per-run hyperparameter files were empty, so the exact realized RF-DETR training settings cannot be fully reproduced. The two model families also use different checkpoint-selection criteria, meaning their reported rows should not be interpreted as perfectly controlled comparisons.

The deployed model is YOLO26-seg nano at 320 px, exported as an OpenVINO intermediate representation. Its accuracy and the reasons for selecting this configuration are reported in Section 3.6.1.

3.3. Stage 3: Perception

Stage 3 converts the live conveyor video into parcel detections. Acquisition uses a single RTSP camera, with TCP preferred and UDP used as a fallback. The stream has a 5 s connection timeout, and a dropped stream is retried every second. Consequently, a temporary network interruption results in lost frames rather than requiring the complete application to restart.

As in System A, a one-slot frame queue retains only the newest decoded frame. Older frames are discarded, preventing processing delays from accumulating when the detector operates below the camera frame rate.

A CLAHE preprocessing stage was implemented as a potential solution to polybag glare. However, it is disabled in the deployed configuration. In the experiment reported in Section 3.6.4, enabling CLAHE repeatedly reduced the number of cartons detected on the same video and configuration. Since this was contrary to the intended effect, the measured behaviour was retained as a deployment result rather than assuming an unverified explanation.

The perception layer supports several inference backends: TensorRT engines, OpenVINO representations, ONNX Runtime graphs, native RF-DETR checkpoints, and Ultralytics checkpoints. The selected backend depends on the available hardware. CPU operation supports YOLO models through OpenVINO or ONNX Runtime, while unsupported RF-DETR files are rejected by OpenVINO because of an identified 2026 toolchain issue with transformer Einsum operators.

The deployed configuration is intentionally minimal: YOLO26-seg nano at a 320 px input, OpenVINO with one inference stream, a confidence threshold of 0.7–0.75, and mask and trace decoding disabled. Only bounding boxes and class labels are required by the trigger logic, so segmentation masks are not decoded at runtime.

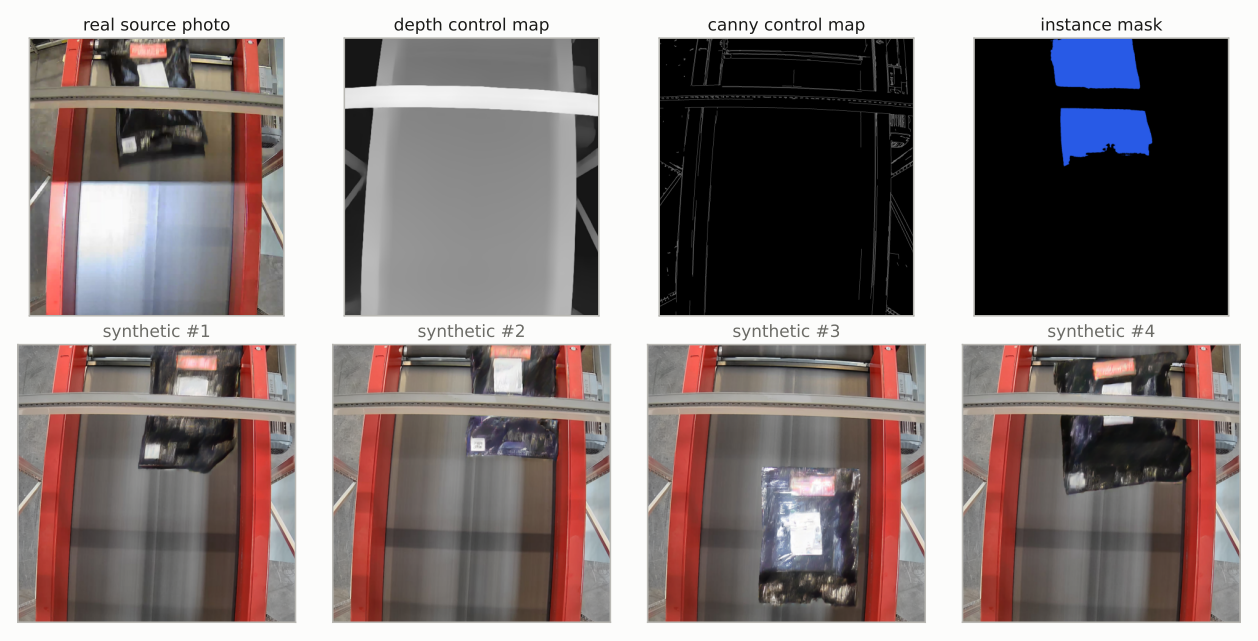

Figure 11. isiGen samples: real photograph, control map and mask scaffold, and synthetic variants with randomized appearance.

3.4. Stage 4: Decision Layer and Parcel Triggering

The decision layer has a different objective from System A. It does not estimate a parcel's metric position. Instead, it determines the precise moment when each parcel crosses a fixed trigger line, assigns its class, and guarantees that exactly one event is generated per parcel.

Tracking for identity. Tracking is used only to maintain temporary parcel identity and prevent duplicate triggers. ByteTrack operates entirely in image coordinates. The tracker is configured using the measured camera/inference frame rate rather than the library's default 30 fps. This is important because its lost-track buffer is expressed in frames. The matching threshold is 0.70 on the CPU path and 0.85 on the GPU path. These values were selected experimentally: overly strict matching caused tracks to break at low processing rates, whereas overly permissive matching could allow a completed track to absorb a following parcel.

Leading-edge anchor. The crossing decision uses the parcel's leading edge rather than its centre. With the deployed top-to-bottom belt direction and horizontal trigger line, this corresponds to the bottom-centre of the bounding box.

This choice was driven by the PLC integration requirement. The customer's timing window is defined relative to the beginning of the parcel (*début de l'objet*). A centre-based trigger would occur later, with the delay depending on parcel length, and could therefore move the event outside the PLC's accepted timing window. Using the leading edge makes the trigger position independent of parcel length and provides a more consistent event time.

Robust crossing detection. At the relatively low inference rates of the deployed system, a parcel can move farther than its own length between two processed frames. A simple test that checks whether the anchor is before the line in one frame and after it in the next can therefore miss a crossing.

To avoid this, Stage 4 uses two complementary crossing tests. The first detects a normal before/after transition. The second implements a gap-tolerant latch: once a track is observed before the line, it becomes armed and fires when the anchor is subsequently observed after the line, even if intermediate frames did not capture the actual crossing. The two tests are combined with an OR operation. If both detect the same crossing, the downstream deduplication gate ensures that only one event is emitted.

Exactly-once event generation. A parcel can trigger only if its track identifier has not already been counted. This prevents repeated events when the parcel remains near the trigger line across several frames. An optional 300 ms time-based guard is available but disabled by default: a fixed time guard could incorrectly suppress two legitimate parcels if they cross the line close together.

Each emitted event receives a monotonic sequence number, incremented only after the deduplication gate accepts the event. The same sequence number is written to the event log. This provides an audit trail: a missing sequence number at the receiver represents an actual lost datagram rather than an event intentionally suppressed by the trigger logic.

The resulting event contains the parcel class and trigger information and is sent once to the sorter interface. This design therefore converts continuous camera detections into a deterministic one-parcel → one-trigger signal suitable for the PLC's timing window.

3.5. Stage 5: Delivery and Deployment

UDP datagram. Each detected crossing generates one UDP datagram of approximately 70 bytes, containing the parcel class, event sequence, tracker ID, and timestamp:

```
{"class": "carton", "seq": 17, "id": 42,
 "ts": "2026-03-31T14:23:45.312847"}
```

The `seq` field is the gap-free event counter generated by Stage 4, while `id` is the ByteTrack identity assigned to the parcel. The two values are therefore intentionally different: `seq` is sequential, whereas `id` is stable for a track but is not necessarily sequential or dense.

The PLC destination is configurable through settings, environment variables, YAML, or a built-in default, and can be changed at runtime. Because the available site documentation contained inconsistent PLC addresses, the destination must be verified during commissioning.

The measured software-side emission time was 78 μs median, 474 μs p99, and 637 μs maximum, at up to approximately 3,000 events/hour. This measures datagram emission from the application; the additional network and PLC-side delay was not measured and is therefore not included in the reported timing.

Relay output and logging. For PLCs that prefer an electrical signal rather than JSON, each crossing can also generate a relay pulse. In the deployed configuration, cartons use channel 1 and polybags

channel 2, with a default pulse duration of 50 ms. Among the supported relay drivers, only the Modbus TCP implementation provides acknowledgement.

Every event is additionally written to a daily CSV log containing timestamp, class, tracker ID, and sequence number. Logs roll over at midnight, are retained for 30 days, and are stored outside the container. The shared sequence number allows transmitted events and recorded events to be compared during troubleshooting.

Operator interface. The system provides a bilingual web interface using the same inference core for both supported backends. Video and runtime statistics are streamed through WebSocket (the live counting view in Fig. 12).

Models can be switched during operation without resetting the event counter or tracker identities. Display encoding is throttled to reduce visualization overhead, while detection, tracking, crossing detection, and event emission continue on every processed frame. Fig. 13 shows the customer-side receiver successfully recording the transmitted datagrams.

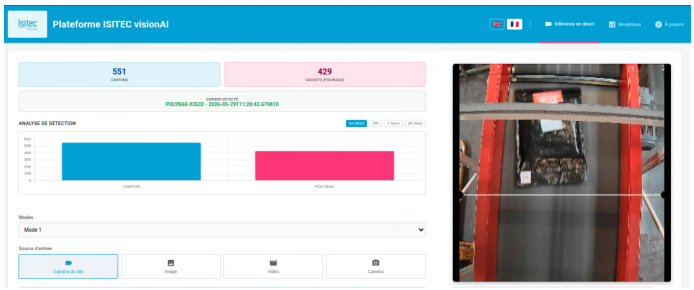

Figure 12. The visionAI platform in live operation: per-class totals, last-detected event, and the belt view with the counting line.

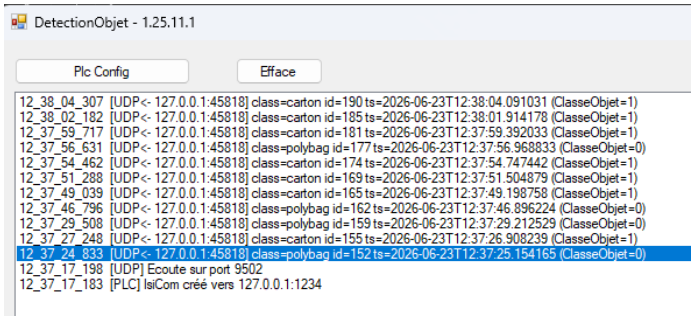

Figure 13. Datagram detail in the customer's receiver: per-parcel class, track id, timestamp, and the `ClasseObjet` flag read by the PLC.

Packaging and deployment. The application is distributed as containers with separate GPU and CPU profiles, requiring approximately 5 GB and 1.2 GB of memory respectively. Hardware selection is performed automatically, with CPU mode used as a fallback when GPU acceleration is unavailable.

A runtime-only deployment branch removes development components from production machines. A typical commissioning procedure takes approximately one hour and includes verification of the three network interfaces, camera connection, remote access, and livestream operation before site handover.

The stack is managed by systemd and starts automatically at boot. This was added after a deployment incident in which a reboot left detection stopped until the application was manually restarted from the browser.

A network lock-down utility freezes the configured network parameters and verifies the UDP path using live packet capture. It also generates a bilingual protocol sheet for the site automation engineer. Model-compression utilities, including FP16 and INT8 conversion, remain part of the office/development workflow and are not required on the deployed system.

The measurements and their limitations are reported in Section 3.6, distinguishing experimentally verified performance from features that have been implemented but not quantitatively evaluated.

3.6. Results for System B

This section evaluates System B in terms of detector performance, synthetic-data contribution, trigger-line selection, counting behaviour, deployment, and model compression.

3.6.1. System B: Parcel Corpus

Training and validation data come from the same recording campaign sampled at 2 fps. Therefore, the results characterize this conveyor and recording conditions rather than generalization to other sites or cameras.

The evaluated models required 3.63–4.73 h of training on the RTX 5070 (Table 1). For RF-DETR, per-class AP was 0.928–0.929 for carton and 0.964–0.972 for polybag, indicating that carton was the more difficult class in these runs.

The deployed YOLO26-seg nano at 320 px was not the most accurate model; the 416 px medium model achieved approximately 0.020 higher box mAP@0.5. However, the 320 px nano model was selected because it satisfies the deployment constraint: CPU-only operation with OpenVINO, which excludes the larger models and RF-DETR.

3.6.2. Stage 2: Synthetic-Data Training Ablation

The contribution of isiGen was evaluated using a polybag-only instance-segmentation ablation with three training conditions: synthetic data only, real data only, and their combination. All models used the same architecture, hyperparameters, and seeds and were evaluated on a common real test set containing 186 images and 215 polybag instances (Table 4).

Synthetic-only training achieved approximately 0.936 box mAP@0.5 on synthetic validation data, but only 0.223 on real images, showing a substantial domain gap.

Combining synthetic and real data improved box mAP@0.5 by 0.021 and recall by 0.050, with a small reduction in precision. However, each condition used a single seed and the test set covered only polybag-positive images from one site. These results therefore indicate a potential benefit from synthetic augmentation but do not establish generalization.

3.6.3. Stage 1: Trigger-Line Position

The trigger-line position was evaluated using three 1,000-frame site clips, producing 35 parcel tracks and 2,609 detections. The leading-edge anchor was highly concentrated at a height fraction of approximately 0.70–0.72, with a median of 0.71 (Fig. 14).

The centre anchor was substantially higher, at approximately 0.45. The evaluation therefore selected 0.71 of the ROI height as the deployed trigger position, using the leading edge as the crossing anchor.

3.6.4. Stage 4: Counting Behaviour

Counting behaviour was compared on one site clip under several configurations. Because the CPU pipeline could not process the original 25 fps video deterministically, the comparison used a fixed frame-decimation procedure.

Tracker-rate calibration, deduplication mode, and interpolation produced identical counts across the tested configurations. CLAHE did affect the result: it produced 19 cartons compared with 23 without CLAHE, a 21 % relative reduction.

All configurations counted zero polybags, despite operator confirmation that polybags were present. Since no labelled ground truth

Table 4. Ablation on the common real test set (186 images, 215 polybag instances).

Arm	Train images (real/syn)	Box P	Box R	Box mAP@0.5	Box mAP@0.5:0.95	Mask mAP@0.5	Mask mAP@0.5:0.95
S (synthetic-only)	0 / 238	0.319	0.391	0.223	0.185	0.219	0.179
R (real-only)	238 / 0	0.922	0.885	0.941	0.927	0.946	0.930
R+S (merged)	238 / 238	0.906	0.935	**0.962**	**0.950**	0.965	0.950

Table 5. Recorded System B sessions (session log on the site machine; the last row ran in the office on a CPU-only host).

Session start	Duration (h)	Mean fps	Carton	Polybag	Mean confidence	Identity ratio
2026-06-03 10:24	0.20	21.8	148	19	not recorded	not recorded
2026-06-16 12:55	2.01	23.0	824	110	0.825	1.95
2026-06-16 16:31	3.17	22.2	1,613	357	0.935	2.41
2026-06-17 09:17	2.69	22.9	1,391	313	0.933	2.41
2026-07-24 10:01	0.02	22.8	9	3	not recorded	not recorded
2026-08-17 16:13 (office, CPU)	0.55	21.5	200	215	0.952	not recorded

The identity ratio is unique tracker identities divided by counted crossings.

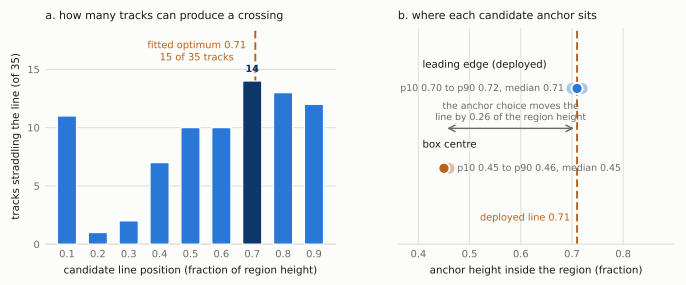

Figure 14. Line-placement study on three site clips: anchor height distributions and straddling tracks; the deployed line sits at 0.71.

exists for the clip, these measurements represent configuration-to-configuration differences rather than counting accuracy. No quantitative counting accuracy is therefore claimed for the deployed pipeline.

3.6.5. Stage 5: Deployed Operation

Multi-hour sessions recorded operation at approximately 22–23 fps with parcel events logged by class (Table 5). A July 2026 event log recorded 12 crossings with sequence numbers 1–12 without gaps, while tracker IDs remained non-sequential, demonstrating the intended separation between the event counter and tracker identity.

A customer-side receiver also successfully logged live UDP datagrams, confirming end-to-end consumption of the communication format by an external system.

However, no formal go-live or acceptance date is recorded. The available evidence demonstrates technical operation but does not establish formal production acceptance.

3.6.6. Stage 5: Compression Artifacts

For the deployed nano model, the measured artifact sizes were: FP32 ONNX 10.96 MB; FP16 5.53 MB (50.5 % of FP32); INT8 OpenVINO 2.78 MB (25.4 % of FP32). One FP16 conversion reproduced the FP32 model's top-five confidences within 2×10^{-4} on a synthetic input.

No systematic post-quantization accuracy evaluation was performed. Therefore, the effect of FP16 or INT8 on detection accuracy and mAP is not reported.

4. DISCUSSION

4.1. System A: Interpretation

Calibration. The 1.176 px calibration residual reported in Table 3 is a bundle-adjustment reprojection error, not a direct measurement of floor-projection accuracy. It is a useful indicator because the same calibrated $(\mathbf{K}, \mathbf{R}, \mathbf{t})$ parameters are used to compute the runtime homography. However, it does not guarantee accuracy in floor areas not covered by the calibration targets. A direct field check using surveyed floor points would therefore be required to fully verify the ≤ 2 px runtime KPI. The August recalibration also shows that fixed cameras can drift and that calibration should be treated as a maintained deployment artifact. The new solve required one session but also required the zone polygons to be redrawn because the world origin changed.

Synthetic data. The isiGen ablation supports using synthetic images as an augmentation rather than a replacement for real data. Synthetic-only training achieved only 0.223 box mAP@0.5 on real images, compared with 0.941 for the count-matched real-data model, indicating a substantial appearance domain gap. Combining real and synthetic data improved mAP by approximately 0.021–0.023 and recall by 0.050, although the mAP improvement is close to the estimated single-seed variation. The experiment is also confounded by the larger amount of training data in the combined condition. A controlled experiment using the same small real dataset with and without synthetic augmentation would provide stronger evidence.

Runtime performance. The final System A configuration achieved 40.3 ms median and 78.1 ms p95 capture-to-publish latency, comfortably below the 200 ms target. This result comes from several combined changes: 720p ingest downscaling, reduced pose input size, motion gating, TensorRT, and the split-process architecture. Their individual contributions were not isolated. The reported latency starts at `capture_ts`, which already includes approximately 100 ms of RTSP buffering and decoding, so it should not be interpreted as optical-event-to-output latency.

Zone crops and 320 px inference. Zone-scoped inference makes the 320 px detector input practical by allocating more pixels to relevant objects while avoiding unnecessary full-frame processing. At 640 px, isolated TensorRT inference improves substantially, but the end-to-end gain is smaller because approximately 41 ms of the 46.7 ms median is spent in CPU-side preprocessing and post-processing. Further gains at larger input sizes would therefore require reducing or accelerating CPU-side processing.

Raised surfaces and rack monitoring. The elevated loading platform demonstrated that a single ground plane is insufficient for raised objects. Introducing a per-zone base height allowed the same ray-plane projection method to handle elevated surfaces without recalibration. Rack monitoring was intentionally kept in image space because the task is a discrete cell-occupancy decision and does not require metric coordinates. This avoids unnecessary calibration and cross-camera complexity and allows racks to be commissioned independently of the geometric system.

4.2. System B: Interpretation

The main challenge in System B was not object detection but reliable event generation. Detector performance was already strong, with

YOLO runs achieving approximately 0.950–0.970 box mAP@0.5. The critical engineering problem was converting frame-level detections into one correctly timed event per parcel.

Four design choices were particularly important:

- **Leading-edge anchor:** matches the PLC timing requirement, which is referenced to the beginning of the parcel.
- **Gap-tolerant crossing latch:** prevents fast parcels from being missed when they cross the line between processed frames.
- **Track-based deduplication:** prevents repeated triggers for the same parcel.
- **Dense sequence counter:** increments only after an event is accepted, making receiver-side gaps attributable to communication loss rather than internal filtering.

The configuration study showed that preprocessing had a larger observed effect on counting than the tested tracking parameters. Disabling CLAHE changed the carton count from 19 to 23 on the same clip, while the tracking-related changes produced identical counts. However, without labelled ground truth, this is only a configuration difference and cannot be interpreted as an accuracy improvement.

The most important unresolved issue is polybag counting. The evaluation clip produced zero polybag events despite operator confirmation that polybags crossed the conveyor, while detector validation showed 0.964–0.972 AP for polybags. The available evidence does not identify whether the problem comes from detection, tracking, or line-crossing logic. Consequently, counting performance cannot currently be considered class-symmetric.

The deployment nevertheless demonstrates practical integration: automatic startup, network verification, bilingual commissioning documentation, persistent event logging, and successful reception by the customer's system. These elements do not constitute research contributions individually, but they demonstrate the transition from an experimental vision pipeline to a deployable industrial system.

4.3. One Core, Two Decision Layers

The two systems share the main perception and deployment infrastructure: detector training, YOLO/RF-DETR models, ONNX export, interchangeable execution providers, preprocessing, logging, configuration, and packaging. The downstream decision layers are fundamentally different because the systems solve different problems (Table 6):

- System A requires metric localization, cross-camera fusion, persistent identity, and zone reasoning.
- System B requires temporal tracking, a leading-edge anchor, line crossing, deduplication, and deterministic event generation.

Therefore, the reusable component is primarily the perception and delivery core, while the decision layer must be designed around the application's operational requirements.

The evidence is also complementary. System A has stronger controlled evaluation, including calibration, latency, inference benchmarks, and training ablations, but lacks customer-side operational evidence. System B has stronger field evidence, including multi-hour operation, customer-side datagram reception, and event logs, but lacks a labelled counting benchmark and extensive automated testing.

Together, the two systems suggest that future industrial deployments should combine both strengths: controlled quantitative evaluation before deployment and structured field validation after deployment.

4.4. Limitations

Several limitations should be considered when interpreting the results.

Table 6. The two systems stage by stage. The first three stages run on shared machinery; the fourth is where the two share nothing.

Stage	System A, warehouse monitoring	System B, conveyor sorting
1 Site geometry	printed boards and floor shots solved into $\mathbf{K}, \mathbf{D}, \mathbf{R}, \mathbf{t}$ and the derived $\mathbf{H}, \mathbf{P}$ per camera (2.1)	one operator rectangle and one counting line, drawn on the image, no camera model (3.1)
2 Data and models	three-class corpus, optional synthetic augmentation, raw-head ONNX (2.2)	two-class parcel corpus, the same export, shipped as an OpenVINO representation (3.2)
3 Perception	one or two cameras, zone-scoped batched crops, pose (2.3)	one camera, operator crop, backends restricted by the host (3.3)
4 Decision layer	**metric geometry**: floor projection, cross-camera fusion, tracking in metres, zone state (2.4)	**event timing**: leading-edge anchor, crossing latch, deduplication, gap-free sequence (3.4)
5 Delivery and deployment	schema-versioned JSON over UDP and MQTT, REST gateway for fleet controllers (2.5)	one datagram and one relay pulse per parcel, containers on the site machine (3.5)

1. **No independent test set.** The detector results are based on validation data, and the reported checkpoints were selected using the same split. The results therefore provide evidence of performance on the evaluated corpus but do not establish independent generalisation.
2. **Limited dataset and deployment diversity.** The experiments were conducted on a limited number of sites, cameras, and operating conditions. Performance under different cameras, lighting, layouts, or operating environments remains unverified.
3. **Limited synthetic-data ablation.** The synthetic-data experiment used one seed, one object class, and relatively small datasets. The synthetic and real-data configurations also used different amounts of training data, making it difficult to isolate the specific contribution of synthetic images.
4. **Limited geometric ground truth for System A.** Calibration was evaluated using reprojection residuals, but no independent field survey or tape-measure campaign was performed to validate runtime floor-projection or 3D accuracy. The reported calibration error should therefore be considered an indirect indicator of deployment accuracy.
5. **Incomplete and partly outdated latency validation.** The reported latency begins at `capture_ts` and excludes approximately 100 ms of camera buffering and decoding. In addition, some execution benchmarks were performed before later deployment changes and were not repeated on the final Jetson target.
6. **No labelled counting ground truth for System B.** The conveyor experiments compare counts obtained under different configurations, but no manually labelled ground truth is available. Consequently, counting precision, recall, miss rate, and false-trigger rate cannot be quantitatively reported.
7. **Hardware and deployment validation.** Most performance measurements were obtained on the RTX 5070/WSL2 development workstation. Although the ONNX-based architecture is designed to support deployment on the Jetson Orin NX, its performance and resource usage were not experimentally benchmarked on the target hardware.

5. CONCLUSIONS

This work presented two industrial vision systems developed around a shared perception and deployment architecture. isiMonitor3d provides metric warehouse monitoring using one or two fixed cameras, while IsiDetector classifies parcels on a conveyor and generates timed sorting events for a PLC. Both systems share the same core components for detection, model export, inference and communication,

while their decision layers are adapted to their respective industrial requirements.

For System A, the main measured acceptance targets were achieved. The system reached a 78.1 ms p95 capture-to-publish latency, below the 200 ms target, a 1.176 px calibration reprojection residual, below the $\leq$ 2 px target, and 0.973–0.977 box mAP@0.5 on the validation split, exceeding the $\geq$ 0.90 target. However, pallet empty/full classification and independent field validation of geometric accuracy remain to be evaluated.

For System B, the deployed detector achieved 0.950 box mAP@0.5 on its validation split. The trigger position was selected experimentally from real conveyor footage, and the system demonstrated multi-hour operation with successful communication to the customer-side receiver. However, the absence of labelled counting ground truth means that counting accuracy and missed-parcel rates could not be quantitatively established.

The main contribution of the work is therefore the integration of reusable perception components with application-specific decision layers rather than the development of a new detection architecture. System A combines operator-guided calibration, ground-plane homography, stereo triangulation, multi-camera identity and metric zones. System B uses image-space tracking, leading-edge crossing detection, identity-based deduplication and gap-free event sequencing. The perception and delivery core was successfully reused across both systems, while the decision logic remained specific to each industrial application.

Future work should focus on strengthening the experimental validation and extending deployment capabilities. For System A, this includes field-based geometric validation, improved synthetic-data experiments, Jetson Orin NX benchmarking, and evaluation with three or more cameras. The rack-cell pipeline also requires live evaluation across additional racks and operating conditions. For System B, priority should be given to establishing labelled counting ground truth, validating trigger timing at the PLC, and quantifying the accuracy impact of FP16/INT8 compression.

Overall, the results demonstrate that a modular and reusable industrial vision architecture can support different monitoring and automation tasks while adapting their decision layers to specific operational requirements. The work also shows that, beyond detector performance, reliable industrial deployment depends strongly on the design and validation of the decision, timing, communication and integration layers.

REFERENCES

[1] Z. Zhang, "A flexible new technique for camera calibration", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000. DOI: 10.1109/34.888718.

[2] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, ISBN: 978-0-521-54051-3.

[3] F. Fleuret, J. Berclaz, R. Lengagne, and P. Fua, "Multicamera people tracking with a probabilistic occupancy map", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 267–282, 2008. DOI: 10.1109/TPAMI.2007.1174.

[4] E. Olson, "AprilTag: A robust and flexible visual fiducial system", in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 3400–3407. DOI: 10.1109/ICRA.2011.5979561.

[5] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion", *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014. DOI: 10.1016/j.patcog.2014.01.005.

[6] E. Bochinski, V. Eiselein, and T. Sikora, "High-speed tracking-by-detection without using image information", in *IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, 2017, pp. 1–6. DOI: 10.1109/AVSS.2017.8078516.

[7] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers", in *European Conference on Computer Vision (ECCV)*, 2020. arXiv: 2005.12872.

[8] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models", in *International Conference on Learning Representations (ICLR)*, 2022. arXiv: 2106.09685.

[9] Y. Zhang *et al.*, "ByteTrack: Multi-object tracking by associating every detection box", in *European Conference on Computer Vision (ECCV)*, 2022. DOI: 10.1007/978-3-031-20047-2_1.

[10] International Organization for Standardization, *ISO 3691-4:2023 — industrial trucks — safety requirements and verification — part 4: Driverless industrial trucks and their systems*, 2023. [Online]. Available: https://www.iso.org/standard/83545.html.

[11] D. Podell *et al.*, *SDXL: Improving latent diffusion models for high-resolution image synthesis*, 2023. arXiv: 2307.01952.

[12] L. Zhang, A. Rao, and M. Agrawala, "Adding conditional control to text-to-image diffusion models", in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. arXiv: 2302.05543.

[13] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, *et al.*, *SAM 2: Segment anything in images and videos*, 2024. arXiv: 2408.00714.

[14] Y. Zhao *et al.*, "DETRs beat YOLOs on real-time object detection", in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv: 2304.08069.

[15] Y. Mori *et al.*, *Multi-camera worker tracking in logistics warehouse considering wide-angle distortion*, 2025. arXiv: 2510.19432. [Online]. Available: https://arxiv.org/abs/2510.19432.

[16] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, *RF-DETR: Neural architecture search for real-time detection transformers*, Accepted at ICLR 2026, 2025. arXiv: 2511.09554.

[17] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, *Ultralytics YOLO26: Unified real-time end-to-end vision models*, 2026. arXiv: 2606.03748.

[18] Microsoft, *ONNX runtime (software), version 1.23.2*, 2026. [Online]. Available: https://onnxruntime.ai/ (visited on 07/20/2026).

[19] NVIDIA Corporation, *AutoMagicCalib: Automated camera calibration for multi-camera 3D tracking*, Part of NVIDIA DeepStream 9.1, 2026. [Online]. Available: https://github.com/NVIDIA-AI-IOT/auto-magic-calib (visited on 08/24/2026).

[20] NVIDIA Corporation, *Multi-view 3D tracking (MV3DT), DeepStream SDK documentation*, 2026. [Online]. Available: https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_MV3DT.html (visited on 08/26/2026).